Модуль №1. Звіт

Ясковець Юлія, ІПЗ-33

OrderFlow: архітектура в умовах змін

1. Підсумок архітектурного рішення

Проблема

OrderFlow є модульним монолітом, який складається з модулів Orders, Payments, Inventory, Notifications і Reporting. Усі модулі працюють в одному процесі, використовують спільну базу даних у пам'яті та розгортаються разом.

Система продовжує працювати, але внесення змін стало повільнішим і дорожчим. Деякі модулі залежать від внутрішньої реалізації інших модулів. Крім того, надсилання сповіщень є частиною процесу оформлення замовлення.

Докази

Інженерні метрики свідчать про погіршення ефективності розробки. Частота випусків зменшилася з п'яти до двох на тиждень, а час доставки змін збільшився з двох до шести днів. Середній розмір pull request зріс зі 180 до 520 рядків коду. Кількість модулів, яких стосується одна функціональна зміна, збільшилася з 1,7 до 4,3.

Частка відкатів зросла з 2% до 9%, а кількість виробничих інцидентів збільшилася з одного до п'яти на місяць. Два нещодавні інциденти почалися під час збільшення затримки платіжного провайдера.

У коді також виявлено прямий доступ Payments до репозиторію Orders. Крім того, Checkout очікує завершення надсилання сповіщення. Такі залежності збільшують вплив змін і можливих збоїв.

Рішення

Доцільно залишити OrderFlow модульним монолітом і поступово вдосконалювати межі між його модулями. Прямий доступ Payments до репозиторію Orders потрібно замінити явним контрактом між модулями. Помилка надсилання сповіщення не повинна перетворювати успішно оплачене та завершене замовлення на невдале.

Контрольований процес резервування товарів слід зберегти в межах оформлення замовлення. Reporting доцільно залишити окремим модулем, а

окрему модель читання запроваджувати лише за наявності підтвердженої проблеми з навантаженням.

Поки що не варто переходити до незалежного розгортання сервісів. Команда складається з восьми розробників і двох QA-інженерів та не має окремої платформної команди.

Компроміси

Модульний моноліт забезпечує відносно просте розгортання й обслуговування. Водночас усі модулі залишаються в одному процесі, використовують спільну базу даних і розгортаються разом.

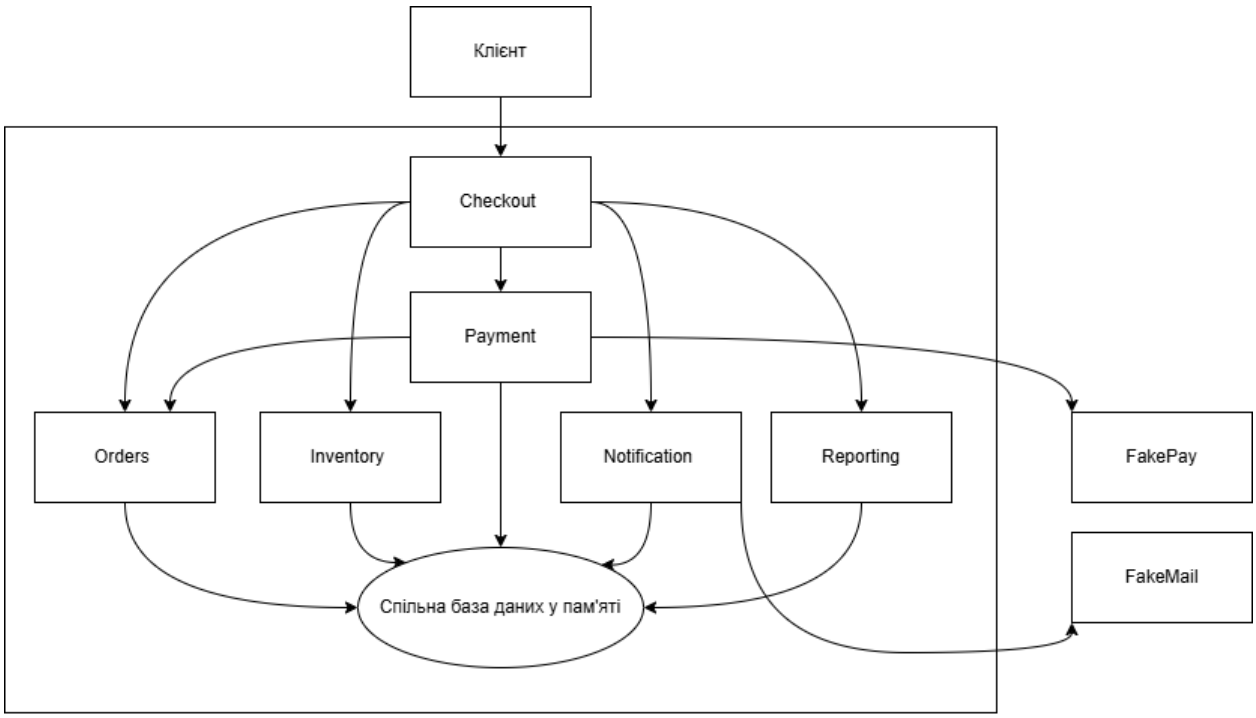
Удосконалення внутрішніх меж зменшить непотрібні залежності, але не усуне необхідність спільного розгортання. Асинхронне надсилання сповіщень могло б підвищити стійкість оформлення замовлення, однак потребувало б додаткової обробки помилок, повторних спроб, моніторингу та контролю доставки повідомлень.

Як вимірюватимемо успіх

Основними показниками будуть час доставки змін, кількість модулів на одну функціональну зміну, частота розгортань, частка відкатів і кількість виробничих інцидентів. Цільові значення потрібно переглянути після встановлення початкових показників оновленої архітектури.

2. Поточна архітектура

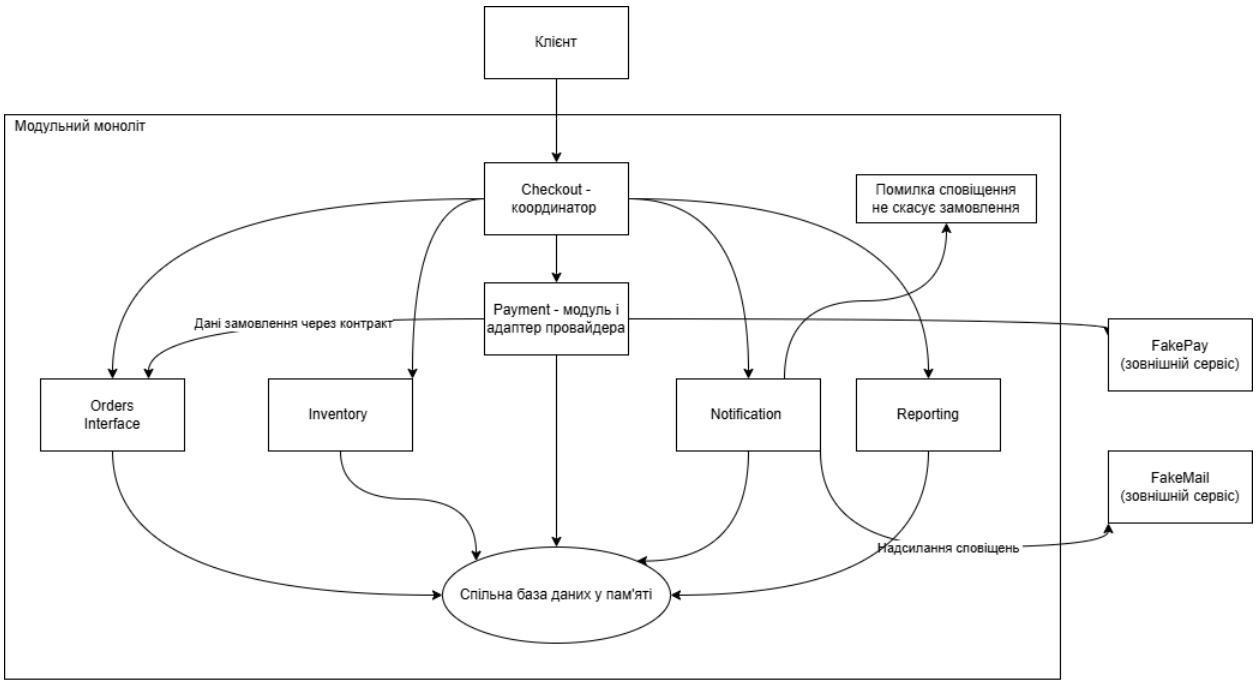
Один процес і одне розгортання



Усі модулі розгортаються разом і використовують спільну базу даних. Пряма залежність Payments від репозиторію Orders розкриває внутрішню реалізацію одного модуля іншому. У початковому процесі Checkout також очікує завершення надсилання сповіщення, тому затримка поштового сервісу може збільшувати час обробки замовлення.

3. Запропонована архітектура

Модульний моноліт: одне розгортання



Запропонований варіант зберігає спільну межу розгортання, але робить контракти між модулями чіткішими. Payments має отримувати необхідні дані

замовлення через визначений інтерфейс Orders, а не звертатися безпосередньо до його репозиторію. Взаємодія з платіжним провайдером повинна залишатися всередині межі Payments.

Помилки сповіщень потрібно обробляти окремо від успішного завершення замовлення. У майбутньому надсилання сповіщень можна перенести до асинхронної черги повідомлень із використанням патерна Outbox. Проте для цього потрібні додаткові механізми надійності, тому така зміна не входить до першого етапу.

Діаграма показує цільову архітектуру. У поточній реалізації ще не додано чергу повідомлень або Outbox.

4. Архітектурні принципи

4.1. Низьке зчеплення

Поточний стан характеризується тим, що Payments безпосередньо звертається до репозиторію Orders. Через це платіжна логіка залежить від внутрішньої структури модуля замовлень.

Рішення полягає у запровадженні визначеного інтерфейсу для отримання необхідних даних замовлення. Це зменшить залежність від конкретної реалізації репозиторію.

4.2. Висока зв'язність усередині модуля

Модулі вже мають окремі бізнес-відповідальності. Orders керує замовленнями, Inventory відповідає за запаси, Payments обробляє платежі, Notifications надсилає сповіщення, а Reporting формує звіти.

Доцільно зберегти ці відповідальності, а не розділяти систему на велику кількість дрібних сервісів. Зміни мають залишатися в тому модулі, якому належить відповідна бізнес-функція.

4.3. Явні контракти

Прямий доступ до репозиторію робить межу між Payments та Orders недостатньо чіткою. Тому потрібно визначити операції та дані, які Orders надає Payments. Інтеграцію з платіжним провайдером слід залишити за адаптером, щоб специфічні деталі провайдера не поширювалися по всій системі.

4.4. Проєктування з урахуванням змін

Шаблони сповіщень змінюються часто, а надсилання листа бере участь у процесі оформлення замовлення. Тому поведінку сповіщень потрібно відокремити від результату оформлення замовлення. Зміна шаблону не повинна вимагати змін в Orders або Inventory. Асинхронне надсилання можна розглянути пізніше, якщо вимірювання підтвердять його доцільність.

5. Пріоритети архітектурних характеристик

Найвищий пріоритет має надійність (Reliability), оскільки результати оплати та резервування товарів повинні залишатися узгодженими. Помилка сповіщення не повинна неправильно скасовувати завершене замовлення.

Високий пріоритет мають супроводжуваність і змінюваність (Maintainability та Modifiability). Межі між модулями повинні зменшувати кількість компонентів, яких стосується звичайна зміна.

Доступність і стійкість до збоїв (Availability та Resilience) також важливі. Затримка зовнішнього платіжного провайдера не повинна спричиняти зайві збої в не пов'язаних із ним частинах системи.

Продуктивність (Performance) залишається важливою характеристикою, але не є головною причиною для повного переписування. Потрібно вимірювати затримки платежів, перш ніж ускладнювати архітектуру розділенням на незалежні сервіси.

Незалежне розгортання та масштабованість (Deployability та Scalability) мають нижчий поточний пріоритет. Наявні докази підтримують спочатку вдосконалення внутрішніх меж, а не перехід до мікросервісів.

Такий порядок пріоритетів свідомо надає перевагу надійності та простоті внесення змін над незалежним розгортанням. Команда має уникати операційних обов'язків, які поки не може ефективно підтримувати.

6. Межі модулів: що змінюємо, а що залишаємо

Межу Payments–Orders потрібно змінити, замінивши прямий доступ до репозиторію явним контрактом. На межі Checkout–Notifications необхідно забезпечити, щоб помилка сповіщення не змінювала успішне замовлення на невдале. Інтеграцію Payments із FakePay потрібно залишити всередині межі Payments.

Взаємодію Orders–Inventory варто зберегти, оскільки вона забезпечує контрольований процес резервування та узгодженість даних. Orders, Payments, Inventory, Notifications і Reporting мають залишитися окремими

модулями в одному застосунку. Reporting потрібно зберегти окремим від процесу оформлення замовлення, а окрему модель читання запроваджувати лише за наявності підтвердженої потреби.

На першому етапі жоден модуль не потрібно розгортати незалежно. Для цього знадобляться надійніші API-контракти, операційний моніторинг, автоматизація розгортання, а також обробка мережевих помилок і визначення власності на дані. У майбутньому можна повторно розглянути Payments або Notifications, якщо потреба в незалежних випусках і очікувана користь виправдають додаткові витрати.

7. Сценарії змін

Сценарій A. Зміна шаблону сповіщення

За запропонованих меж зміна шаблону має обмежуватися модулем Notifications. Orders та Inventory не повинні потребувати змін коду. Потрібно виконати тести сповіщень і скорочену регресійну перевірку інтеграції з Checkout.

Сценарій B. Додавання правила оплати

Правило потрібно реалізувати всередині Payments, використовуючи контракт Orders для отримання необхідних даних замовлення. Тестування має охоплювати платіжну логіку, інтеграцію з провайдером і межу взаємодії з Checkout.

Сценарій C. Зростання навантаження на звітність

Reporting потрібно залишати окремим від відповідальностей оформлення замовлення. Спочатку слід виміряти затримки звітів та їхній вплив на застосунок. Якщо навантаження стане підтвердженим вузьким місцем, можна додати окрему модель читання або інше цільове рішення замість розділення всіх модулів.

8. Результати невеликого покращення

У Checkout реалізовано невелику зміну: помилка надсилання сповіщення обробляється окремо й не перетворює завершене оплачене замовлення на невдале.

Додано регресійний тест, який перевіряє, що в разі помилки сповіщення замовлення залишається завершеним, оплата — успішною, а товар — зарезервованим. Усі шість тестів проходять успішно.

Це обмежене покращення, а не повноцінне асинхронне надсилання сповіщень. Checkout усе ще може очікувати на поштовий сервіс, тому його затримка залишається потенційною проблемою продуктивності.

Висновок

OrderFlow поки не потребує повного переписування або переходу на мікросервіси. Наявні докази обґрунтовують поступову еволюцію архітектури: зменшення прямих залежностей, уточнення контрактів і відокремлення помилок сповіщень зі збереженням модульного моноліту та простішої моделі розгортання.

Подальші зміни потрібно визначати за вимірюваними показниками швидкості доставки, надійності та операційних потреб. Незалежне розгортання варто розглядати лише тоді, коли його переваги перевищуватимуть додаткову складність.